

Multi-Agent Academic Operating System (MAAOS)

Requirement Analysis and Feasibility Study

1. Requirement Analysis

1.1 Functional Requirements

ID	Requirement
FR1	Connect to a student's Gmail account via OAuth and the Gmail API
FR2	Use LLMs to extract assignments, exam dates, deadlines, and project reviews from emails
FR3	Auto-generate tasks from extracted information without manual entry
FR4	Prioritize tasks based on deadline proximity and importance
FR5	Assess student workload and adjust scheduling accordingly
FR6	Generate a personalized day-wise study/task schedule
FR7	Send reminders/notifications before deadlines
FR8	Extract text from uploaded documents/images via OCR
FR9	Answer student queries from uploaded notes using RAG
FR10	Track and display academic progress/analytics
FR11	Authenticate users via JWT-based login
FR12	Switch between cloud and local LLMs depending on connectivity

1.2 Non-Functional Requirements

Category	Requirement
NFR1 — Reliability	Offline fallback via local Ollama models when cloud API is unavailable
NFR2 — Performance	Email parsing and task creation within a few seconds of arrival
NFR3 — Security	JWT authentication, encrypted storage of Gmail OAuth tokens
NFR4 — Scalability	Multi-agent architecture allows independent scaling of agents
NFR5 — Usability	PWA accessible on desktop and mobile without app-store install
NFR6 — Maintainability	Modular agent design (CrewAI) for independent updates
NFR7 — Portability	Dockerized deployment across environments

1.3 Hardware Requirements

Component	Specification
Server	Minimum 8GB RAM, 4-core CPU (GPU optional, recommended for local LLM inference)
Client	Any device with a modern browser (PWA)

1.4 Software Requirements

Component	Technology
Frontend	React (PWA)
Backend	FastAPI, CrewAI
Database	PostgreSQL/MySQL, ChromaDB (vector store)
OCR	Tesseract
LLM orchestration	LiteLLM, Ollama
Containerization	Docker
Auth	JWT

2. Feasibility Study

Technical Feasibility

All required technologies (Gmail API, LLMs, CrewAI, ChromaDB, Tesseract, Docker) are mature, well-documented, and open-source or freely accessible, making the system technically achievable with existing skills and infrastructure. The main technical risk is LLM extraction accuracy on varied/noisy email formats, mitigated by the hybrid router and RAG grounding.

Economic Feasibility

Core components (FastAPI, React, PostgreSQL, ChromaDB, Ollama, Tesseract) are open-source with no licensing cost. The main recurring cost is cloud LLM API usage, which is minimized by routing routine requests to a local Ollama model. Overall cost is low relative to the productivity benefit for students.

Operational Feasibility

The system replaces manual task tracking with automation, requiring minimal behavior change from students beyond granting Gmail access once. Since it works through PWA (no install) and offline-capable local models, day-to-day operation is straightforward.

Schedule Feasibility

Given a modular multi-agent design, individual agents (Email Intelligence, OCR, Reminder, etc.) can be developed and tested in parallel, making the project achievable within a typical academic project timeline (one semester).

Legal/Ethical Feasibility

Gmail API access requires OAuth consent and handling of personal data (emails) responsibly — requiring secure token storage and a clear privacy policy, since email content includes personally identifiable academic information.